

# Assignment 4 : Machine Learning Fundamentals

## Group 3

Members: Aditya Singh, Krishnapriya Reddy, Poonam Gupta

### Question 1: Why is Machine Learning needed instead of traditional programming?

Traditional programming works well when rules are fixed and clearly defined. However, many real-world problems like image recognition, speech processing, or fraud detection do not have explicit rules. Machine Learning allows systems to learn patterns directly from data and improve with experience. This makes ML more flexible, scalable, and effective for complex and dynamic problems.

**Traditional Programming:**  $\text{Input} + \text{Rules} \rightarrow \text{Output}$

**Machine Learning:**  $\text{Input} + \text{Output} \rightarrow \text{Model (Rules)}$

### Question 2: Differentiate between supervised, unsupervised, and reinforcement learning.

- **Supervised Learning:** The model learns from labeled data (input + correct output).  
**Example:** Linear Regression, Classification

$\text{Input} + \text{Label} \rightarrow \text{Model}$

- **Unsupervised Learning:** The model finds hidden patterns from unlabeled data.  
**Example:** K-Means clustering, PCA

$\text{Input} \rightarrow \text{Patterns}$

- **Reinforcement Learning:** The model learns by interacting with an environment and receiving rewards or penalties.  
**Example:** Q-learning, Deep Q Networks

$\text{State} \rightarrow \text{Action} \rightarrow \text{Reward} \rightarrow \text{Policy}$

### Question 3: Least squares solution for linear regression

Linear regression aims to minimize the sum of squared errors between predicted and actual values. The optimal weights are obtained by minimizing the cost function, which leads to the closed-form solution.

For univariate linear regression:

$$y_i = wx_i + b$$

$$J(w, b) = \sum_{i=1}^n (y_i - (wx_i + b))^2$$

Taking partial derivatives:

$$\frac{\partial J}{\partial w} = -2 \sum_{i=1}^n x_i (y_i - wx_i - b) = 0$$

$$\frac{\partial J}{\partial b} = -2 \sum_{i=1}^n (y_i - wx_i - b) = 0$$

Solving these equations gives:

$$w = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2}$$

$$b = \bar{y} - w\bar{x}$$

where  $\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$  and  $\bar{y} = \frac{1}{n} \sum_{i=1}^n y_i$ .

For multivariate linear regression, the model can be written in matrix form as:

$$\mathbf{y} = X\boldsymbol{\theta}$$

and minimizing the squared error leads to the normal equation:

$$\boldsymbol{\theta} = (X^T X)^{-1} X^T \mathbf{y}$$

However, this closed-form solution requires the matrix  $X^T X$  to be invertible. In practical scenarios, this condition may not hold due to multicollinearity among features, redundant variables, or when the number of features exceeds the number of observations. As a result, the inverse  $(X^T X)^{-1}$  may not exist.

To overcome this limitation, the Moore–Penrose pseudoinverse is used. The pseudoinverse of the design matrix  $X$ , denoted by  $X^+$ , generalizes matrix inversion and exists for any matrix, irrespective of its shape or rank. Using the pseudoinverse, the least squares solution can be expressed as:

$$\boldsymbol{\theta} = X^+ \mathbf{y}$$

This formulation yields the least squares solution that minimizes  $\|X\boldsymbol{\theta} - \mathbf{y}\|^2$  and selects the minimum-norm solution among all possible solutions. When  $X^T X$  is invertible, the pseudoinverse reduces to:

$$X^+ = (X^T X)^{-1} X^T$$

thereby recovering the standard normal equation.

In the general case, the pseudoinverse is computed using Singular Value Decomposition (SVD), which ensures numerical stability and guarantees a valid and robust solution even when the design matrix is singular or non-square.

## Question 4: What is gradient descent?

Gradient descent is an iterative optimization algorithm used to minimize a loss function. In linear regression, it updates weights step-by-step in the direction of steepest descent to reduce prediction error. It is especially useful when datasets are large and closed-form solutions are expensive.

For linear regression:

$$\frac{\partial J}{\partial w} = -\frac{2}{n} \sum_{i=1}^n x_i (y_i - wx_i - b)$$
$$\frac{\partial J}{\partial b} = -\frac{2}{n} \sum_{i=1}^n (y_i - wx_i - b)$$

Update rules:

$$w \leftarrow w - \alpha \frac{\partial J}{\partial w}$$
$$b \leftarrow b - \alpha \frac{\partial J}{\partial b}$$

General form:

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \alpha \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$$

## Question 5: Explain L1 and L2 regularization

Regularization is used to reduce overfitting by adding a penalty term to the loss function.

**L1 Regularization (Lasso):**

$$J(\boldsymbol{\theta}) = \text{Loss}(\boldsymbol{\theta}) + \lambda \sum_{j=1}^p |\theta_j|$$

L1 regularization penalizes the absolute value of the weights, forcing some weights to become zero and hence performing feature selection.

**L2 Regularization (Ridge):**

$$J(\boldsymbol{\theta}) = \text{Loss}(\boldsymbol{\theta}) + \lambda \sum_{j=1}^p \theta_j^2$$

L2 regularization penalizes the squared weights, shrinking all weights and improving model stability.

L1 removes unimportant features, whereas L2 reduces weight magnitude to prevent overfitting.

## Question 6: Regularization to reduce overfitting

Regularization adds a penalty term to the loss function, discouraging overly complex models. This helps the model generalize better to unseen data rather than memorizing training samples by constraining the model's capacity.

## Question 7: Why can't linear regression be used for classification?

Linear regression is used to predict continuous values, whereas classification requires discrete class labels.

The linear regression model is given by:

$$\hat{y} = \mathbf{w}^T \mathbf{x} + b$$

The output of this model is unbounded  $(-\infty, +\infty)$  and therefore cannot represent class probabilities limited to  $[0, 1]$ .

Linear regression minimizes mean squared error, which is not suitable for classification and is sensitive to outliers. Hence, linear regression cannot reliably model decision boundaries for classification problems, and classification algorithms such as logistic regression are preferred.

## Question 8: Logistic regression model

Logistic regression is a supervised learning algorithm used for binary classification problems. It predicts the probability that a given input belongs to a particular class.

A linear combination of the input features is first computed:

$$z = \mathbf{w}^T \mathbf{x} + b$$

This value is passed through the sigmoid (logistic) function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

The logistic regression model is therefore given by:

$$P(y = 1 \mid \mathbf{x}) = \frac{1}{1 + e^{-(\mathbf{w}^T \mathbf{x} + b)}}$$

The predicted class is obtained using a threshold:

$$\hat{y} = \begin{cases} 1, & \text{if } P(y = 1 \mid \mathbf{x}) \geq 0.5 \\ 0, & \text{otherwise} \end{cases}$$

Logistic regression uses the binary cross-entropy (log loss) function:

$$J(\mathbf{w}, b) = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

## Question 9: Sigmoid function and its importance

The sigmoid function  $\sigma(z) = \frac{1}{1+e^{-z}}$  maps values into the range  $(0,1)$ . This makes it ideal for probability interpretation in binary classification tasks. Its smooth gradient prevents jumps in output values, and it's differentiable, which is essential for gradient-based optimization.

## Question 10: Gradient descent in logistic regression

The logistic regression model computes:

$$z_i = \mathbf{w}^T \mathbf{x}_i + b$$

$$\hat{y}_i = \sigma(z_i) = \frac{1}{1 + e^{-z_i}}$$

The binary cross-entropy loss function is:

$$J(\mathbf{w}, b) = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

The gradients are:

$$\frac{\partial J}{\partial \mathbf{w}} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i) \mathbf{x}_i$$
$$\frac{\partial J}{\partial b} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)$$

Gradient descent update rules:

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \frac{\partial J}{\partial \mathbf{w}}$$
$$b \leftarrow b - \alpha \frac{\partial J}{\partial b}$$

## Question 11: Clustering vs classification

**Clustering** is an unsupervised learning technique that groups data points into clusters such that points within the same cluster are more similar to each other than to those in other clusters. No labeled data is required.

**Classification** is a supervised learning technique in which data points are assigned to predefined class labels using labeled training data.

| Aspect                   | Clustering               | Classification       |
|--------------------------|--------------------------|----------------------|
| Unsupervised             | Supervised               | Learning Type        |
| Yes                      | Labels Required          | No                   |
| Objective                | Discover hidden patterns | Predict class labels |
| Clusters                 | Class labels             | Output               |
| Logistic Regression, SVM | Examples                 | K-Means, DBSCAN      |

## Question 12: K-Means algorithm steps

K-means is an unsupervised learning algorithm used to partition data into  $K$  clusters.

1. Choose the number of clusters  $K$ .

2. Initialize  $K$  centroids randomly from the data points.
3. Assign each data point to the nearest centroid using Euclidean distance:

$$\text{Cluster}(x_i) = \arg \min_j \|x_i - \mu_j\|^2$$

4. Update each centroid by computing the mean of all data points assigned to that cluster:

$$\mu_j = \frac{1}{|C_j|} \sum_{x_i \in C_j} x_i$$

5. Repeat steps 3-4 until convergence (centroids no longer change significantly).

### Question 13: Limitations of K-Means

- Requires specifying  $K$  in advance
- Sensitive to initial centroid selection (may converge to local optima)
- Assumes clusters are spherical and of similar size
- Sensitive to outliers (mean is influenced by extreme values)
- Only works with numerical data and Euclidean distance
- Not suitable for clusters with non-convex shapes

### Question 14: Choosing the value of K

- **Elbow Method:** Plot within-cluster sum of squares (WCSS) against  $K$ :

$$\text{WCSS}(K) = \sum_{j=1}^K \sum_{x_i \in C_j} \|x_i - \mu_j\|^2$$

Choose  $K$  at the "elbow point" where rate of decrease slows.

- **Silhouette Method:** Compute silhouette score for each  $K$ :

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}$$

Choose  $K$  with highest average silhouette score.

- **Domain Knowledge:** Use prior knowledge about the data structure.
- **Gap Statistic:** Compare WCSS with expected WCSS under null distribution.

### Question 15: Overfitting and underfitting

- **Overfitting:** Model learns noise and details from training data, performs well on training but poorly on test data. High variance.
- **Underfitting:** Model is too simple to capture patterns, performs poorly on both training and test data. High bias.

## Question 16: Bias–variance tradeoff

The bias–variance tradeoff describes the balance between underfitting and overfitting.

- **Bias:** Error from overly simplistic assumptions in the model
- **Variance:** Error from sensitivity to fluctuations in training data

The expected error decomposes as:

$$\mathbb{E}[(y - \hat{f}(x))^2] = \text{Bias}^2(\hat{f}(x)) + \text{Var}(\hat{f}(x)) + \sigma^2$$

where  $\sigma^2$  is irreducible error.

## Question 17: Train-test split

Data is divided into training and testing sets to evaluate the model's performance on unseen data. Typical splits are 70-30 or 80-20. This prevents information leakage and provides an unbiased evaluation.

## Question 18: K-fold cross-validation

K-fold cross-validation divides the dataset into  $K$  equal-sized folds. The model is trained on  $K - 1$  folds and validated on the remaining fold. This process repeats  $K$  times with each fold serving as validation once.

The final performance is the average across all folds:

$$\text{CV Score} = \frac{1}{K} \sum_{i=1}^K \text{Score}_i$$

This provides a more reliable estimate than a single train-test split.

## Question 19: Precision, recall & confusion matrix

A confusion matrix evaluates classification performance:

|                 | Predicted           |                     |
|-----------------|---------------------|---------------------|
|                 | Positive            | Negative            |
| Actual Positive | True Positive (TP)  | False Negative (FN) |
| Actual Negative | False Positive (FP) | True Negative (TN)  |

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \quad (\text{How many selected items are relevant?})$$

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \quad (\text{How many relevant items are selected?})$$

$$\text{F1-score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

## Question 20: Importance of data preprocessing

Data preprocessing improves model performance by:

- Handling missing values and outliers
- Removing noise and inconsistencies
- Scaling features to similar ranges
- Encoding categorical variables
- Reducing dimensionality
- Ensuring compatibility with algorithm assumptions

## Question 21: Handling missing data

- **Deletion:** Remove rows/columns with missing values (if few missing)
- **Imputation:** Fill missing values with:
  - Mean/median/mode
  - Interpolation
  - Model-based prediction
  - K-Nearest Neighbors
- **Indicator:** Add binary flag for missingness

## Question 22: Categorical encoding technique

One-hot encoding converts categorical variables into binary vectors:

For categorical variable *Color* with values {Red, Blue, Green}:

| Color | Red | Blue | Green |
|-------|-----|------|-------|
| Red   | 1   | 0    | 0     |
| Blue  | 0   | 1    | 0     |
| Green | 0   | 0    | 1     |

Alternative: Label encoding (for ordinal data), Target encoding (for high cardinality).

## Question 23: Importance of feature scaling

Feature scaling is crucial because:

- Distance-based algorithms (KNN, K-means) are sensitive to feature magnitudes
- Gradient-based algorithms converge faster with scaled features
- Prevents features with large ranges from dominating
- Improves numerical stability



## Question 24: Standardization vs normalization

- **Standardization (Z-score):**

$$x' = \frac{x - \mu}{\sigma}$$

Result: mean = 0, variance = 1. Good for most algorithms.

- **Normalization (Min-Max):**

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

Result: range [0,1]. Good for bounded algorithms like neural networks.

## Question 25: Poor learning rate issues

- **Too high:** Overshooting, divergence, unstable updates
- **Too low:** Slow convergence, may get stuck in local minima
- **Adaptive methods:** Adam, RMSprop adjust learning rate automatically

## Question 26: Batch vs Mini-batch vs SGD

Let loss for sample  $i$  be  $\ell_i(\theta)$ , total loss:

$$J(\theta) = \frac{1}{n} \sum_{i=1}^n \ell_i(\theta)$$

- **Batch Gradient Descent:** Uses all samples

$$\theta \leftarrow \theta - \alpha \frac{1}{n} \sum_{i=1}^n \nabla \ell_i(\theta)$$

Stable but slow for large datasets.

- **Stochastic Gradient Descent:** Uses one random sample

$$\theta \leftarrow \theta - \alpha \nabla \ell_i(\theta)$$

Fast but noisy updates.

- **Mini-batch Gradient Descent:** Uses  $m$  samples (typically 32-512)

$$\theta \leftarrow \theta - \alpha \frac{1}{m} \sum_{i \in B} \nabla \ell_i(\theta)$$

Best balance between stability and speed.

## Question 27: Feature engineering

Creating and transforming features to improve model performance:

- Polynomial features
- Interaction terms
- Binning/discretization
- Logarithmic/power transformations
- Text feature extraction (TF-IDF, embeddings)
- Time-based features

## Question 28: Neural networks and their need

Neural networks are computational models inspired by biological neural systems. They can learn complex non-linear relationships and hierarchical representations.

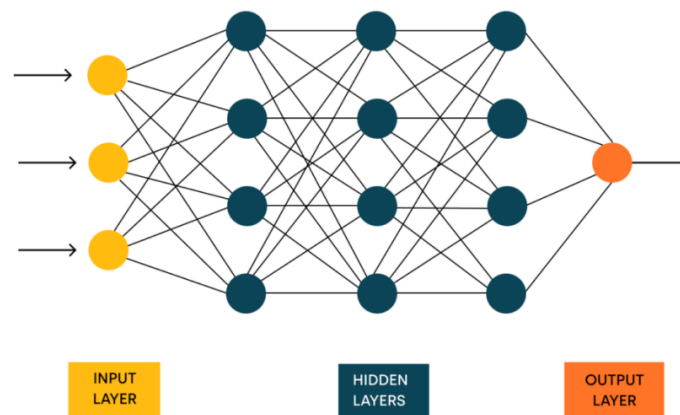


Figure 1: Neural Network Architecture

Neuron computation:

$$z = \sum_{i=1}^n w_i x_i + b, \quad a = f(z)$$

They excel at tasks with high-dimensional data (images, text, audio) where traditional models struggle.

## Question 29: Perceptron model

The perceptron is a binary linear classifier:

$$z = \sum_{i=1}^n w_i x_i + b$$

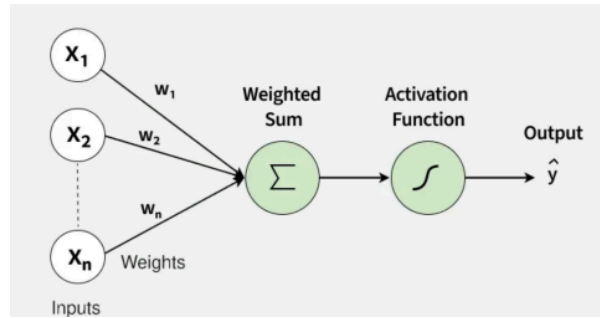


Figure 2: Perceptron Model

$$\hat{y} = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

Learning rule:

$$w_i \leftarrow w_i + \eta(y - \hat{y})x_i$$

Limitation: Cannot learn non-linearly separable functions like XOR.

### Question 30: Importance of activation functions

Activation functions introduce non-linearity, enabling neural networks to approximate any continuous function (Universal Approximation Theorem). Without them, multiple layers would be equivalent to a single linear layer.

### Question 31: ReLU vs Sigmoid

- **Sigmoid:**  $\sigma(x) = \frac{1}{1+e^{-x}}$ 
  - Range: (0,1)
  - Smooth gradient
  - Suffers from vanishing gradients
  - Output not zero-centered
- **ReLU:**  $\text{ReLU}(x) = \max(0, x)$ 
  - Range:  $[0, \infty)$
  - Computationally efficient
  - Mitigates vanishing gradient
  - May cause dead neurons

ReLU is preferred in hidden layers; sigmoid/tanh in output layers for classification.

### Question 32: Feedforward neural network

Information flows forward without cycles:

$$\begin{aligned}\mathbf{a}^{(0)} &= \mathbf{x} \\ \mathbf{z}^{(l)} &= \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)} \\ \mathbf{a}^{(l)} &= f^{(l)}(\mathbf{z}^{(l)})\end{aligned}$$

for  $l = 1, \dots, L$ , where  $L$  is number of layers.

### Question 33: Backpropagation (intuitive)

Backpropagation computes gradients efficiently using chain rule:

1. Forward pass: Compute activations and loss
2. Backward pass: Compute gradients layer by layer
3. Update weights using gradients

### Question 34: Why neural networks overfit easily

Neural networks overfit because:

- High capacity (many parameters)
- Can memorize noise in training data
- Complex architectures learn intricate patterns

Prevention methods:

- Regularization (L1/L2)
- Dropout
- Early stopping
- Data augmentation
- Batch normalization